

Vector Space Day SF — Research & Strategy Briefing

Event: Vector Space Day SF, 2026-06-11, The Midway **Speakers/sponsors of note:** LlamaIndex, mem0, Neo4j, Google DeepMind, Qualcomm, Arize, TwelveLabs **Tracks:** Search & AI Retrieval · Agents & Memory · Edge & Robotics AI

This document captures deep research on (1) harness engineering, (2) the AI agent memory landscape, (3) how BrainOS fits, (4) a futuristic missing-piece pitch, and (5) a direct comparison to Graphiti.

1. What is Harness Engineering, and why the shift?

The one-line definition

Agent = Model + Harness

The **harness is everything around the model except the model itself**: the orchestration loop, the tool layer, context management, long-term memory, verification gates, observability, and guardrails.

Mitchell Hashimoto's canonical operating principle:

"Every time you discover an agent has made a mistake, you take the time to engineer a solution so that it can never make that mistake again."

That "solution" lives in the harness.

The three-generation evolution

Era	Unit of control	What you tune
Prompt engineering (2022–2024)	The message	Wording, few-shot examples, chain-of-thought
Context engineering (2025)	The session	What goes into the window — RAG, tool defs, history
Harness engineering (2026)	The system	The runtime loop around N inferences

Dick Lightowler's mental model: the model is the CPU, the harness is the OS. Context

Rick Hightower's mental model: the model is the CPU, the harness is the OS. Context engineering is one *layer* inside the harness — not its replacement.

Why the shift was forced (the data)

1. **65% of enterprise AI project failures trace back to harness-level data defects** rather than model reasoning (Masood, *AI Control Plane*).
2. **Identical model + identical data + identical prompts → 42% to 78% success rates** purely based on runtime environment quality (Epsilla).
3. **10x cost reduction is achievable from harness changes alone:** \$3.00/MTok → \$0.30/MTok plus 4x latency drop, *without changing the model* — through prefix stability, semantic caching, and KV-cache locality (Masood).
4. **TerminalBench:** harness changes moved agents **20+ ranking positions** without touching weights (Daily Dose of DS).

The ETCLOVG canonical taxonomy (7 layers)

Layer	What it does	Examples
E — Execution loop	Observe → Think → Act, when to stop	Claude Code agent loop, AutoGPT loops
T — Tool/function calling	Capability surface area	MCP servers, function definitions, logits masking
C — Context management	Window discipline	Compaction, summarization, prefix-cache stability
L — Long-term memory/state	Cross-session persistence	Scratchpads, AGENTS.md, todo.md, virtualized memory
O — Orchestration	Multi-step flow control	Retries, max-step caps, DAG routing, supervisors
V — Verification	Quality gates	Plan-Execute-Verify, generator-evaluator pairs
G — Guardrails & observability	Safety + telemetry	Approval gates, secret masking, audit trails, tracing

Patterns worth knowing

- **The "Orientation Tax"** — tokens spent because the agent doesn't have a structural map of the environment, leading to "grep-sprees." The fix is **environment engineering**: restructure your repo, docs, and APIs to be *legible* to agents.
- **Plan-Execute-Verify with model layering** ("reasoning sandwich") — expensive reasoning model plans and verifies; cheap model does intermediate work. Becoming the default loop

topology.

- **MCP & A2A protocols** — MCP is the "USB-C for AI" vertical agent-to-tool standard; A2A is horizontal delegation across frameworks.
-

2. AI Agent Memory Landscape — Market + OSS Scan

The market has split into **four architectural philosophies**, each with a flagship.

A) Middleware fact-extraction — Mem0

- **Architecture:** Library/SDK between app and vector DB. Every `add()` calls an LLM to extract facts, stores them as discrete entries with user/session/agent scoping.
- **Strengths:** Lowest setup cost, great for personalization chatbots, broadest framework coverage (LangChain, LangGraph, LlamaIndex, CrewAI, AutoGen).
- **Weakness:** LLM call on every write → latency. Graph features paywalled.
- **Benchmark:** LongMemEval 49.0% (GPT-4o).

B) Temporal knowledge graph — Zep (commercial) / Graphiti (OSS)

- **Architecture:** Full memory **server** with async summarization, entity extraction, and a temporal knowledge graph storing timestamped facts + relationship maps. Understands *state changes*: "I lived in London → moved to Tokyo" becomes a temporally-edged transition.
- **Strengths:** Best for enterprise SaaS where user state evolves. Async write path keeps reads fast.
- **Benchmark:** LongMemEval 63.8% — a **15-point lead over Mem0** driven entirely by the temporal graph.
- **Weakness:** Heavier ops footprint, more concepts.

C) OS-style self-managed — Letta (formerly MemGPT)

- **Architecture:** Agent itself owns its memory. Core memory (fixed-size, always in context) + archival memory (searchable). LLM uses tool calls to read/write/promote/demote.
- **Strengths:** Genuine autonomy. Agents that run for *days* and curate their own working set.
- **Weakness:** High setup complexity. Latency penalty because the agent itself spends tokens managing memory.

D) GraphRAG — Cognee

- **Architecture:** Builds a structured knowledge graph from unstructured text. Multi-hop reasoning across documents.
- **Strengths:** Multi-document corpora. deep retrieval over corpus relationships.

- **Weakness:** Specialized; overkill for chat-style personalization.

Emerging contrarian: Verbatim / no-summarization — MemPalace

- **Insight:** every summarization step is lossy and one LLM-call away from hallucination cascade. Don't summarize, don't extract — just store every conversation in Chroma and retrieve.
- **Benchmark:** Hit **96.6% on LongMemEval** in raw-storage mode.
- **Counter-argument:** storage costs and retrieval slowdown at scale.

The Tulving taxonomy (LangMem crystallized it)

Production memory systems converge on three types:

- **Episodic** — specific past events ("On 2026-03-12, the user said X")
- **Semantic** — facts and preferences ("user prefers Postgres over MySQL")
- **Procedural** — agent's own learned operating instructions, often as self-rewriting system prompts

Benchmarks that matter in 2026

- **LoCoMo** — 1,540 questions, multi-session recall. Top score: **92.5**.
- **LongMemEval** — 500 questions, knowledge updates + temporal + multi-session. Top score: **94.4**.
- **BEAM** — 1M to 10M token production scale. **Performance drops ~25% from 1M (64.1) to 10M (48.6)** — the field has a *scale wall*.

The six unsolved problems (Mem0's own 2026 report)

1. **Temporal abstraction at scale** — degrades 25% between 1M → 10M token corpora
2. **Cross-session structure** — systems treat change as *replacement* rather than *evolution*
3. **Application-level eval** — benchmarks don't predict domain workload performance
4. **Privacy architecture** — no standard for consent / retention / deletion
5. **Cross-session identity** — assumes stable `user_id` ; breaks on multi-device / anonymous
6. **Memory staleness** — high-relevance memories become *confidently wrong* after circumstances change

Two deeper gaps nobody is shipping

- **"Half-solved problem" (Moses Njau)** — current systems store *what happened*, not *what was learned from what happened*. Diary entries, not notebook insights. Example: a log says "increased pool 10→20, CPU went up 15%, timeout unresolved" — but the *lesson* "increasing pool size hurts this service" is what should live in memory.

- **Session amnesia (OSS Insight)** — "Every architecture decision you debated, every bug you traced, every shortcut you explained — gone." No system has demonstrated *longitudinal* value (months, not benchmark runs).

3. Graphiti deep-dive (the engine under Zep)

Critical context: Graphiti is the OSS engine *underneath* Zep.

- **Graphiti** = open-source temporal knowledge graph library you self-host (`github.com/getzep/graphiti`)
- **Zep** = Graphiti + multi-tenancy + managed infra + dashboards + SLAs

"Graphiti is the open-source core engine; Zep adds managed infrastructure, multi-user/conversation management, pre-configured retrieval, dashboards, and enterprise SLAs."

What Graphiti uniquely does

1. **Bi-temporal model** — every edge has *two* time axes: when the event **occurred** (validity window) and when it was **ingested**. Lets you ask "what did we know last Tuesday?" *and* "what was actually true last Tuesday?"
2. **Invalidation, not deletion** — old facts marked invalid with `valid_to` . History preserved.
3. **Episodes as provenance unit** — raw ingested chunks are first-class nodes linked to every extracted fact.
4. **Hybrid retrieval** — semantic embeddings + BM25 + graph traversal.
5. **Real-time incremental** — no batch recompute on update; entities resolve immediately against existing nodes.
6. **Multi-backend** — Neo4j (default), FalkorDB, Kuzu (embedded), Amazon Neptune.
7. **Pydantic ontology** — custom entity/edge types as typed Python classes.
8. **Peer-reviewed** — arXiv 2501.13956, ICLR 2026 MemAgents Workshop.

Cross-system comparison

	Mem0	Letta	Cognee	MemPalace	Gr
Storage shape	Vector entries	Tiered (core+archival)	GraphRAG	Verbatim vector	Real graph DE
Temporal model	timestamp field	None first-class	Limited	None	Bi-temporal v windows
Fact invalidation	Overwrite	Manual	Limited	N/A	Soft-invalidat

Update model	Per-add LLM call	Agent-driven	Batch	Append	Real-time inc
Backend	Vector DB	DB + buffer	Vector + graph	Chroma	Neo4j/FalkorDB
Provenance	Metadata	Limited	Source links	Raw text	Episodes as n

4. Can BrainOS use harness engineering?

Yes — and the more interesting framing is that BrainOS is *already partially* a harness for other people's agents, while internally it under-uses harness patterns for itself.

What BrainOS already does well (mapped to ETCLOVG)

Harness layer	BrainOS today	Strength
T — Tools	<code>brainos_agent/tools.py:1</code> exposes 10 read/write tools (<code>ask_brain</code> , <code>search_facts</code> , <code>lookup_entity</code> , <code>get_relationships</code> , <code>ingest_text</code> ...)	Strong — this <i>is</i> a tool surface for external agents
C — Context	<code>SKILLS.md</code> / <code>SKILLS.json</code> is a structured context bundle	Strong — agents load a curated subset, not raw docs
V — Verification	<code>FeedbackAgent</code> audits groundedness, triggers revision when confidence <0.72	Strong — closes the loop on the <i>answer</i> , not the corpus
G — Guardrails	<code>_is_sensitive()</code> blocklist, <code>EXPORT_TOKEN</code> gate, <code>disputed=true</code> flag	Partial
L — Long-term memory	<code>brain.json</code> + ChromaDB + BM25 + entity index + graph	Strong — but passive

The thing BrainOS does that the rest of the field doesn't: it solves the **Orientation Tax**. Most memory systems give the agent raw chunks. BrainOS gives the agent a *legible map* — atomic units, typed entities, directed verbs, confidence scores, temporal status. This is exactly what Masood means by "environment engineering" and it's a real moat.

Where BrainOS is weak in harness terms

Harness layer	Gap	Concrete fix
E — Execution loop	The four agents are linear (ingest → struct → exec → feedback). No retry budget, no max-step caps, no escalation policy	Add a runtime ledger: per-job token cap, retry count, escalation to human when FeedbackAgent fails twice
O — Orchestration	No supervisor over the agents. If StructuringAgent's LLM reconciliation verdict is uncertain, nothing arbitrates	Add a Supervisor agent that gets called only when sub-agent confidence < threshold
L — Memory for the brain itself	Brain stores facts about the <i>company</i> , but nothing about <i>its own behavior</i> — which retrievals worked, which extractions later got reconciled away, which sources keep producing low-confidence units	Add meta-memory : a tiny self-graph about which signals/sources/agents performed well per question class
V — Verification	FeedbackAgent only audits one answer. Nothing audits ingestion ("did this unit prove useful in any answer that scored well?")	Outcome-link units to answers — see §5
G — Observability	<code>_call_log</code> ring buffer is good. But no eval harness — no "we should have answered X" regression set	Add a golden Q&A set auto-extracted from past resolved Slack threads

Concrete near-term harness wins for BrainOS (ranked by ROI)

1. **MCP server wrapping** `brainos_agent/tools.py` — turns BrainOS into a first-class tool source for any agent runtime (Claude Code, Cursor, Devin). Tool definitions exist; wrapper is ~200 lines.
2. **Per-source confidence calibration** — `_log_call()` already tracks calls; extend to track which `source_id` led to grounded vs. ungrounded answers. Re-weight retrieval per source.
3. **Plan-Execute-Verify on ingestion**, not just on Q&A — before merging a contested unit, plan the reconciliation, execute the merge in a shadow brain, verify downstream queries don't regress against a golden set.
4. **Budget ledger** — every API route gets a token/call cap with explicit fail-fast.
5. **Agent-readable repo structure for *your customers*** — generate `AGENTS.md`, `ARCHITECTURE.md`, `OWNERS.md` from `brain.json` so any external coding agent can orient in seconds. The Orientation Tax fix as a *product*.

5. The Missing Piece — Futuristic Pitch (three layers)

Every memory startup is shipping a **better filing cabinet**. The opportunity is to ship the **first brain that has opinions about itself**.

Idea A — Outcome-Linked Memory (ship this quarter)

One-line pitch: RLHF for the knowledge layer, not for the model.

The gap it closes: Mem0's #6 (staleness). Njau's "other half" (lessons not events).

How it works:

- Every unit gets two new fields: `usefulness_score` (decayed EMA of contributions to grounded answers) and `last_used_at`.
- Every Q&A in `routes/ask.py` emits a closed-loop signal back to `brain.json`: `"unit u_42 was cited; FeedbackAgent scored answer 0.91 → reward unit u_42."`
- ChromaDB scoring boost picks up `usefulness_score` at retrieval time.
- RRF weights become *learned* per question class (`core/indexes.py:71`), not fixed at `k=60`.
- Units that haven't been used in 90 days AND have `confidence < 0.7` get auto-flagged for the gap analyzer.

Why defensible: Static knowledge graphs become *living* knowledge graphs. The longer BrainOS runs, the smarter it gets — which competitors can't claim because they don't have a closed FeedbackAgent loop.

Implementation footprint: ~300 lines. Touches `agents/execution.py`, `agents/feedback.py`, `storage/brain.py`, `core/indexes.py`. Two days of work.

Idea B — The Belief Layer (new SKU)

One-line pitch: Beneath facts sits a separate graph of *causal hypotheses* the brain holds with degrees of confidence, actively triangulates, and is willing to retire.

The gap it closes: Njau's "other half" *fully* (causal lessons as first-class objects). Mem0's "cross-session structure" (change as evolution).

Architecture:

```
FACTS (brain.json units)
"We chose Stripe in Q2 2024"
```

| async triangulation



```
BELIEFS (new beliefs.json)
hypothesis: "Stripe is preferred
  for SMB payments because of
  SCA support"
confidence: 0.74 (Bayesian)
support: [u_4, u_19, u_88]
contradicts: [u_201]
half_life: 180d
last_triangled: 2026-04-10
```

| when confidence drops



```
ACTIVE QUERIES (Slack DMs)
"@alice - I'm seeing 3 cases
  where pool>20 hurt latency.
  Pattern or coincidence?"
```

Three new agents:

- **DreamAgent** — runs on idle CPU/GPU cycles. Reads recent units, proposes new beliefs by clustering causal patterns. This is **memory consolidation** — the gap nobody addresses (the literal "sleep" pattern from human cognition).
- **TriangulationAgent** — for each belief, finds independent corroborating sources. A belief supported by N independent sources gets a confidence boost; one supported by N restatements of the same source gets penalized.
- **InquisitorAgent** — when belief confidence drops below threshold, *actively asks humans on Slack to confirm/deny*. Human-in-the-loop active learning at the belief layer, not the fact layer.

Why it's a category move: Mem0/Zep/Letta/Cognee/Graphiti are all **passive databases**. A belief layer is the first **epistemic agent** — a memory system that has *opinions about its own knowledge*. The pitch deck line: "Your company brain doesn't just remember — it *suspects, doubts, and learns*."

What makes it shippable from BrainOS:

- Existing schema (`confidence` , `evidence` , `disputed` , `validFrom` / `validTo` , `supersededBy`) is 97% *there*.
- Add `beliefs.json` with same provenance/temporal/confidence pattern.
- DreamAgent is just a scheduled job hitting the LLM with "here's the diff of new units; what causal patterns do you propose?"

Idea C — Federated Tacit Memory (12-month bet)

One-line pitch: Slack threads are 60% of company knowledge; nobody can share them across companies because of privacy; but the *patterns* in those threads — how teams resolve incidents, which architectural debates produce what outcomes — are universally valuable. Federate the patterns, not the data.

Why now (market signal): Interloom raised **\$16.5M** in Mar 2026 doing exactly the single-company version of this (calls it a "context graph"). The federated version is unbuilt.

The mechanic:

- Each company's BrainOS instance produces **anonymized belief signatures**: "When `kind=ownership` units about `system=payments` were contradicted 3+ times within 30 days, 78% correlated with an upcoming reorg."
- Signatures aggregated across orgs into a **shared belief market** — public-good knowledge layer of "things that tend to be true across companies."
- Local brains query the market for **priors** when their own belief confidence is low.
- Privacy: only typed patterns flow out; never units, never entities, never quotes.

Why it's futuristic and uncontested:

- Solves Mem0's gap #4 (privacy architecture) by *requiring* a privacy boundary at the design level.
- It's the **transactive memory** pattern (Wegner 1985) applied across organizations, not within one team.
- No one is doing it because everyone is competing for raw-data possession. Contrarian bet: raw data stays sovereign and *meta-knowledge* federates.

Ranking for action

1. **For Vector Space Day conversations:** lead with **Idea B (Belief Layer)** — cleanest crystallization of every gap in the field. BrainOS's existing schema is the closest thing in OSS to a belief-ready foundation.
2. **For roadmap:** ship **Idea A (Outcome-Linked Memory)** this quarter — cheap, directly demos the "brain that learns" pitch, sets up data foundation for Idea B.
3. **For fundraising story 12 months out:** **Idea C (Federated Tacit Memory)** — Interloom's raise validates the single-org category; federation is the moat-building move.

6. BrainOS vs Graphiti — Direct Comparison

BrainOS and Graphiti share a *lot* of philosophy — explicit graph, temporal facts, hybrid retrieval, provenance. Differences are real but specific.

Capability	Graphiti	BrainOS
Graph backend	Real graph DB (Neo4j/FalkorDB/Kuzu/Neptune) — Cypher queryable	<code>brain.json</code> + ChromaDB + in-memory index. Simpler, but no Cypher / no graph algorithms
Temporal model	Bi-temporal: <code>occurred_at</code> + <code>ingested_at</code> per edge, explicit validity windows	Single-temporal: <code>validFrom</code> / <code>validTo</code> on units. Adds <code>temporalStatus</code> ∈ {current, expired, historical} as an explicit enum
Conflict handling	Soft-invalidation: old edge gets <code>valid_to</code>	Richer: <code>supersededBy</code> , <code>conflictsWith</code> , <code>disputed=true</code> , <code>disputedReason</code> , structured <code>disputedReason</code> , explicit verdicts from <code>StructuringAgent</code> (<code>supersedes</code> / <code>duplicate</code> / <code>conflicts</code>)
Ingestion	Episodes → entities + edges	Sources → entities + units (with <code>kind</code> fact/process/decision/ownership/definition + relationships. Typed atomic units as abstraction.
Multi-modal	Text only	VLM pipeline (Qwen2.5-VL / LLaVA) — whiteboards, screenshots → graph edges
Groundedness audit	None	FeedbackAgent — audits every answer
Knowledge-gap analysis	None first-class	Deterministic graph traversal surfaces ownership/coverage
Agent surface	Python library; you build the agent	MCP-style tool surface in <code>brainos_api</code> 10 tools, designed to be called <i>by</i> other agents
Output artifact	The graph itself	<code>brain.json</code> + <code>SKILLS.md</code> (agent-loadable per department) + live <code>/metrics</code>
Positioning	Memory engine for developers to embed	End-to-end product (UI, ingest workflow, dashboards)
Scope	Per-user / per-context graph	Org-level company brain

Where Graphiti beats BrainOS

- **Real graph database** — Cypher queries, graph algorithms (PageRank, community detection, shortest path) are free.
- **Bi-temporal correctness** — the "what did we know on date X" question is built-in. BrainOS's `temporalStatus` enum is coarser.
- **Scale** — Neo4j/FalkorDB scale to billions of edges; `brain.json` won't.
- **Peer-reviewed architecture** — arXiv paper carries credibility.

Where BrainOS beats Graphiti

- **Typed unit kinds** (fact/process/decision/ownership/policy/gotcha) — Graphiti only has entities + edges.
- **Explicit conflict reconciliation** — Graphiti invalidates; BrainOS classifies the *reason* (supersedes vs. conflicts vs. duplicate) via LLM verdict.
- **Multi-modal** — Graphiti is text-only.
- **FeedbackAgent / groundedness audit** — closed loop on answers; Graphiti has no answer layer at all.
- **End-to-end product** — UI, ingest flows, conflict surfacing, metrics. Graphiti is a library; BrainOS is an app.

Strategic implication

Graphiti is the most credible competitor on the *storage layer*, but BrainOS plays a different game. Two responses:

1. **Borrow** — Graphiti's bi-temporal model (`occurred_at` + `ingested_at` as separate fields per edge) is a clean upgrade to the single-axis temporal fields. A few hours of work, immediate epistemological win.
2. **Pivot the conversation** — when asked "why not just use Graphiti?", the answer is: *Graphiti is the substrate; we're the operating system on top*. The right move might literally be to swap `brain.json` for a **Graphiti backend** and keep everything above — you'd get a real graph DB *and* preserve all differentiation.

BrainOS-on-Graphiti could be a stronger product than either alone. Graphiti handles temporal+graph mechanics at scale; BrainOS keeps the agent-shaped product layer and typed knowledge model. Worth a 1-day spike to see how much of `storage/brain.py` could become a Graphiti adapter.

Sources

Harness Engineering

- [Harness engineering for coding agent users — Martin Fowler](#)
- [Agent Harness Engineering — The Rise of the AI Control Plane \(Masood\)](#)
- [The Anatomy of an Agent Harness \(Avi Chawla\)](#)
- [The Third Evolution: Why Harness Engineering Replaced Prompting in 2026 \(Epsilla\)](#)
- [Harness Engineering vs Context Engineering \(Hightower\)](#)
- [What Is an Agent Harness? \(Firecrawl\)](#)

Memory Layer Approaches

- [State of AI Agent Memory 2026 \(Mem0\)](#)
- [Mem0 vs Zep vs Letta vs Cognee 2026 \(n1n.ai\)](#)
- [The Agent Memory Race of 2026 — 5 Repos, 4 Architectures \(OSS Insight\)](#)
- [The Memory Problem in AI Agents Is Half Solved \(Njau\)](#)
- [Multi-Agent Shared Graph Memory \(Neo4j\)](#)
- [Best AI Agent Memory Frameworks 2026 \(Atlan\)](#)

Graphiti

- [Graphiti GitHub \(getzep/graphiti\)](#)
- [Graphiti: Temporal Knowledge Graphs for Agentic Apps \(Zep Blog\)](#)
- [Zep: A Temporal Knowledge Graph Architecture for Agent Memory \(arXiv 2501.13956\)](#)
- [Graphiti: Knowledge Graph Memory for an Agentic World \(Neo4j\)](#)
- [Graphiti Open Source — Zep](#)

Market Signal

- [Interloom raises \\$16.5M for tacit-knowledge context graph \(Fortune\)](#)